

C Programming

Project Structure & Build Guide

A complete guide for students with no prior C experience

Bank Management System — Academic Edition
Version 1.0 | 2026

1. What Is a C Project?

A C project is a collection of plain text files that end in `.c` (source files) and `.h` (header files). Unlike languages such as Python, C code must be compiled — transformed from human-readable text into a binary executable — before it can run. This document walks you through every step of that process from an empty folder to a working program.

NOTE C is one of the oldest and most widely used programming languages. It runs in operating systems, embedded devices, databases, and game engines. Learning C teaches you how computers actually work.

2. File Types and Their Roles

Before creating a single file you must understand what each type is for.

Extension	Name	Purpose
<code>.c</code>	Source file	Contains the actual code — functions, logic, algorithms.
<code>.h</code>	Header file	Declares function names and data types shared between <code>.c</code> files. Never put executable code here.
<code>.o</code>	Object file	Intermediate compiled form of one <code>.c</code> file. Produced during compilation, deleted after linking.
binary	Executable	The final program. On Linux it has no extension (e.g. <code>atm</code>); on Windows it is <code>.exe</code> .
Makefile	Build script	Plain text file that automates compilation. Running 'make' reads it and executes the right compiler commands.

3. Naming Conventions

3.1 Source Files (`.c`)

Use lowercase letters and underscores only. Each file should contain one cohesive set of related functions.

Pattern	Example
<code><module>.c</code>	<code>auth.c</code> — authentication functions
<code><module>_utils.c</code>	<code>hashing_utils.c</code> — utility helpers for hashing
<code>get_valid_<input>.c</code>	<code>get_valid_phone.c</code> — input validation for phone numbers

<component>_menu.c	init_menu.c — the initialisation / welcome menu
main.c	Always the entry point — contains the main() function

3.2 Header Files (.h)

Every .c file that exposes functions to other files should have a matching .h file with the same base name. The header acts as a contract — it tells other files what functions exist without showing the implementation.

```
/* validators.h */
#ifndef VALIDATORS_H      /* include guard — prevents double inclusion */
#define VALIDATORS_H

int val_phone(const char *input, char err_out[128]);
int val_amount(const char *input, double *out, char err_out[128]);

#endif /* VALIDATORS_H */
```

RULE Always wrap header files in an include guard (`#ifndef` / `#define` / `#endif`). Without it, if two files include the same header, the compiler sees duplicate declarations and exits with an error.

3.3 Variables and Functions

Identifier type	Convention	Example
Local variables	snake_case	int account_nbr;
Global variables	g_ prefix	static char g_pending_notif[1024];
Public functions	verbNoun	createNewAcc(), makeTransaction()
Private (static) functions	_verbNoun	static void _label(WINDOW *w, int y, ...);
Constants / macros	ALL_CAPS	#define VALIDATOR_ERR_BUF 128
Struct names	PascalCase	struct Record, struct User, struct Date

4. Folder and Directory Structure

Organise files into sub-directories grouped by responsibility. Flat projects become unmanageable beyond 10 files.

```
project/
├── main.c           # Entry point
```

```

├── header.h           # Shared structs and global declarations
├── auth.c / auth.h    # Authentication module
├── ui_helper.c         # UI utility functions
├── system.c           # Core banking operations
├── init_menu.c        # Login / register screen
├── main_menu.c        # Main navigation menu
├── ui/                # TUI (terminal user interface) layer
│   ├── tui.c
│   └── tui.h
├── utils/             # Reusable low-level helpers
│   ├── hashing_utils.c / .h
│   ├── ipc_utils.c / .h
│   ├── string_utils.c / .h
│   ├── file_utils.c / .h
│   └── terminal_utils.c / .h
├── validation/        # Pure input validators
│   ├── validators.c
│   ├── validators.h
│   ├── get_valid_account.c
│   ├── get_valid_amount.c
│   ├── get_valid_country.c
│   ├── get_valid_date.c
│   └── get_valid_phone.c
├── database/          # Database initialisation and helpers
│   ├── database.c / .h
│   └── database_oper.c
├── data/              # Runtime data (not compiled)
│   ├── atm.db         # SQLite database (auto-created)
│   └── countries.txt   # Country lookup file

```

TIP The data/ folder is never compiled. It holds files your program reads and writes at runtime. Never put .c or .h files there.

5. The #include System

When one .c file needs to call a function defined in another .c file, it must #include the corresponding .h file. This tells the compiler the function signature before the linker connects everything together.

```

/* system.c needs validators – it includes their header */
#include "../validation/validators.h"

/* Use angle brackets for system / library headers */
#include <stdio.h>
#include <sqlite3.h>
#include <ncurses.h>

/* Use double quotes for YOUR OWN headers */
#include "../header.h"

```

```
#include "../ui/tui.h"
```

RULE Angle brackets < > are for libraries installed on the system. Double quotes " " are for your own project files. The path inside quotes is relative to the current file's location.

6. Compiling a C Program

6.1 The Compiler: GCC

GCC (GNU Compiler Collection) is the standard C compiler on Linux. It turns .c source files into an executable binary through two phases: compilation (each .c becomes a .o object file) and linking (all .o files and libraries are joined into one binary).

6.2 Compiling a Single File (learning exercise)

```
# Compile one file into an executable called 'hello'
gcc hello.c -o hello

# Run it
./hello
```

6.3 Compiler Flags You Must Know

Flag	Meaning
-o <name>	Name the output binary. Without this, gcc produces a file called 'a.out'.
-Wall	Enable all standard warnings. Treat every warning as a hint of a bug.
-Wextra	Enable extra warnings beyond -Wall. Use both together.
-g	Embed debug information so you can use GDB to step through code.
-I<path>	Add a directory to the header search path. e.g. -I./utils
-L<path>	Add a directory to the library search path. e.g. -L/usr/local/lib
-l<lib>	Link a library. e.g. -lsqlite3 -lcurses -lpthread -lcrypto
-std=c11	Compile using the C11 standard (recommended for modern C).

6.4 Compiling a Multi-File Project Manually

When a project has many .c files, list them all on the command line:

```
gcc -Wall -Wextra -g \
    main.c auth.c system.c init_menu.c main_menu.c ui_helper.c \
```

```

ui/tui.c \
utils/hashing_utils.c utils/ipc_utils.c utils/terminal_utils.c \
utils/string_utils.c utils/file_utils.c \
validation/validators.c validation/get_valid_account.c \
validation/get_valid_amount.c validation/get_valid_country.c \
validation/get_valid_date.c validation/get_valid_phone.c \
database/database.c database/database_oper.c \
-o atm \
-lsqlite3 -lncurses -lmenu -lpthread -lssl -lcrypto

```

NOTE The backslash `\` at the end of each line is a line continuation character — it tells the shell that the command continues on the next line. This is purely for readability.

7. Automating Builds with a Makefile

Typing the full `gcc` command every time is error-prone. A Makefile stores it permanently. Run 'make' to recompile only the files that changed.

7.1 Minimal Makefile Structure

```

# Makefile
CC      = gcc                                # compiler
CFLAGS  = -Wall -Wextra -g -std=c11          # compile flags
LDFLAGS = -lsqlite3 -lncurses -lmenu -lpthread -lssl -lcrypto
TARGET  = atm                                # output binary name

# List every .c file
SRCS = main.c auth.c system.c ...

# Derive .o names automatically
OBJS = $(SRCS:.c=.o)

# Default target: build the binary
all: $(TARGET)

$(TARGET): $(OBJS)
$(CC) $(OBJS) -o $(TARGET) $(LDFLAGS)

# Rule for any .c -> .o
%.o: %.c
$(CC) $(CFLAGS) -c $< -o $@

# Remove compiled files
clean:
rm -f $(OBJS) $(TARGET)

```

IMPORTANT The indentation before each command in a Makefile **MUST** be a real Tab character, not spaces. This is one of the most common mistakes beginners make.

7.2 Running Make

```
make          # compile the project
make clean    # delete all .o files and the binary
make clean all # full rebuild from scratch
```

8. Installing Library Dependencies

C programs that use external libraries (like SQLite or ncurses) require those libraries to be installed on the system before compilation.

8.1 Ubuntu / Debian (apt)

```
sudo apt update

# SQLite (database)
sudo apt install libsqlite3-dev

# ncurses (terminal UI)
sudo apt install libncurses-dev

# OpenSSL (hashing)
sudo apt install libssl-dev

# pthreads is included with glibc - no separate install needed
```

8.2 macOS (Homebrew)

```
brew install sqlite ncurses openssl

# On macOS you may need to tell gcc where to find headers:
export CFLAGS="-I$(brew --prefix)/include"
export LDFLAGS="-L$(brew --prefix)/lib"
```

9. Running the Binary

After a successful 'make', the executable sits in your project root. Always run it from the project root directory, because relative paths in the code (like ./data/atm.db) are resolved from wherever you launch it.

```
# Always run from the project root
cd /path/to/project
./atm

# Do NOT run from another directory
# /path/to/project/atm <- this may fail to find data/atm.db
```

10. Debugging

10.1 Compiler Warnings Are Your First Debugger

Always compile with `-Wall -Wextra`. A warning from the compiler is the compiler telling you there is almost certainly a bug. Fix every warning before asking why the program crashes.

10.2 GDB — The GNU Debugger

```
# Compile with debug symbols (the -g flag)
gcc -g -Wall main.c -o atm ...

# Start the debugger
gdb ./atm

# Inside GDB:
(gdb) break main           # set a breakpoint at the start of main()
(gdb) run                  # start the program
(gdb) next                 # execute one line
(gdb) print variable_name # inspect a variable's value
(gdb) backtrace            # show the call stack after a crash
(gdb) quit                 # exit GDB
```

10.3 Valgrind — Memory Error Detector

```
# Install
sudo apt install valgrind

# Run your program through valgrind
valgrind --leak-check=full ./atm

# Output tells you about memory leaks, invalid reads/writes
```

TIP If your program crashes with 'Segmentation fault', run it under valgrind or GDB immediately. A segfault almost always means you accessed memory you do not own — a null pointer, an out-of-bounds array index, or a freed pointer.

11. Common Beginner Mistakes

Mistake	Fix
Missing include guard in .h	Add <code>#ifndef MY_HEADER_H / #define / #endif</code>
Tabs vs spaces in Makefile	Use only Tab for rule indentation, never spaces
Putting code in .h files	Declarations only in .h. Implementation goes in .c
Forgetting <code>-l</code> flags at link	Link errors say 'undefined reference'. Add the missing <code>-l</code> flag

Using gets() or scanf("%s")	Both overflow buffers. Use fgets() with a size limit instead
Comparing strings with ==	Use strcmp(a,b)==0. == compares pointer addresses, not content
Not initialising variables	Always memset structs to 0 after declaring them
Running binary from wrong dir	cd into the project root first, then ./binary

12. New Project Checklist

1. Create the directory structure: src/, utils/, ui/, data/, database/
2. Write main.c with int main(void) as the entry point
3. Create header.h with shared struct definitions and function prototypes
4. Add each module as a .c + .h pair with matching include guards
5. Install library dependencies with apt/brew
6. Write the Makefile listing all .c files and the correct -I flags
7. Run make. Fix every warning the compiler emits.
8. Run ./binary from the project root
9. Compile with -g and use valgrind to catch memory errors
10. Run make clean before committing to version control

FINAL NOTE

Always read compiler error messages from top to bottom and fix the first error first. Later errors are often cascading consequences of the first one, and they will disappear on their own once the root cause is resolved.